Operating Systems

Sheet 6 Solutions

Concurrency: Monitors, Message Passing, Readers/Writers

Name: Yehia Said Gewily

ID: 21010705

Date: April 2026

1 Semaphores cont. & Monitors (5.4 + 5.5)

1. Interleaving Analysis of Two Concurrent Processes

Two processes $P1$ and $P2$ execute the same code on the shared variable `x`. Initially, `x = 10`. Each statement such as `x = x + 1` or `x = x - 1` is not atomic. It is executed in three machine steps:

`LD R0, x` → `INCR/DECR R0` → `STO R0, x`

Because of interleaving, strange outputs may appear.

Part a — A sequence that prints "x is 10"

Step	P1	P2	x
1	Executes `x = x - 1` completely	—	9
2	Starts `x = x + 1`, loads 9 into register, then stops	—	9
3	—	Checks `if (x != 10)` and finds it true	9
4	—	Stops before `printf`	9
5	Resumes and finishes `x = x + 1`	—	10
6	—	Executes `printf("x is %d", x)`	10

So the condition was checked while `x = 9`, but the printed value became 10 before `printf` read it.

Part b — A sequence that prints "x is 8"

Step	P1	P2	x
1	Executes `x = x - 1` completely	—	9
2	Starts `x = x + 1`, loads 9, then stops	—	9
3	—	Executes `x = x - 1` completely	8
4	—	Checks `if (x != 10)` and prints `x`	8

So $P2$ sees `x = 8` and prints it before $P1$ restores the old value.

2. Add Semaphores so that `printf()` Never Executes

To make sure `printf()` is never executed, the two statements

```
x = x - 1;
x = x + 1;
```

must be executed atomically as one critical section.

We use one binary semaphore initialized to 1:

```
shared int x = 10;
Semaphore s = 1;

while (1) {
    semWait(s);
    x = x - 1;
    x = x + 1;
    semSignal(s);

    if (x != 10) {
        printf("x␣is␣%d", x);
    }
}
```

Why this works:

- Only one process can update `x` at a time.
- After the decrement and increment finish, `x` returns to 10.
- Therefore the condition `x != 10` is always false, so `printf()` never runs.

This gives the required protection while keeping the critical section as short as possible.

3. Difference Between Semaphores and Monitors

Although both semaphores and monitors use the ideas of `wait` and `signal`, they are different.

Aspect	Semaphore	Monitor
Type	Low-level synchronization tool	High-level language construct
Mutual exclusion	Must be handled manually by programmer	Built in automatically
Wait/Signal	Operate on an integer counter	Operate on condition variables
Ease of use	Harder and easier to misuse	Safer and easier to understand
Signal when nobody waits	Signal is saved in semaphore value	Signal is lost if no process is waiting

Summary: Semaphores are lower-level and depend on correct use by the programmer. Monitors are higher-level and automatically provide mutual exclusion.

2 Message Passing (5.6)

4. Message Passing and Mutual Exclusion

Message passing is a communication method in which processes exchange information by sending and receiving messages.

The basic operations are:

- `send(destination, message)`
- `receive(source, message)`

Instead of sharing memory directly, processes communicate through these messages.

Using message passing for mutual exclusion:

A single message can be used as a **token**. Whoever has the token is allowed to enter the critical section.

```
while (true) {
    receive(mutex, msg);   // get the token
    /* critical section */
    send(mutex, msg);      // return the token
    /* remainder section */
}
```

Why this works:

- There is only one token.
- So only one process can receive it at a time.
- Therefore only one process can enter the critical section at a time.

3 Readers/Writers Problem (5.7)

5. Problem with the Given Readers/Writers Implementation

The main problem in this implementation is **writer starvation**.

In this solution, readers are given priority:

- The first reader locks `wrt`.

- Other readers can continue entering as long as at least one reader is still reading.
- A writer must wait until all readers leave.

If new readers keep arriving continuously, then `readcount` may never become 0. So the writer may wait forever.

Conclusion: The solution allows multiple readers correctly, but it is unfair to writers because a writer can starve.

6. Readers/Writers Solution Using a Monitor

A monitor-based solution can be written as follows:

```
monitor ReaderWriter {
    int readcount = 0;
    bool writing = false;
    condition canRead, canWrite;

    procedure startRead() {
        if (writing)
            canRead.wait();
        readcount = readcount + 1;
        canRead.signal();
    }

    procedure endRead() {
        readcount = readcount - 1;
        if (readcount == 0)
            canWrite.signal();
    }

    procedure startWrite() {
        if (writing || readcount > 0)
            canWrite.wait();
        writing = true;
    }

    procedure endWrite() {
        writing = false;
        canWrite.signal();
        canRead.signal();
    }
}
```

Reader process:

```
ReaderWriter.startRead();
/* reading */
ReaderWriter.endRead();
```

Writer process:

```
ReaderWriter.startWrite();
/* writing */
ReaderWriter.endWrite();
```

Idea:

- Readers can enter together if no writer is active.
- Writers must wait until there are no readers and no writer.
- The monitor automatically protects shared variables.

4 General Questions

7. Lamport's Bakery Algorithm

Part a — Description in words

Lamport's Bakery Algorithm works like taking numbers in a bakery shop.

1. A process that wants to enter the critical section chooses a number.
2. It chooses a number bigger than all current numbers.
3. Then it waits until all processes with smaller numbers finish first.
4. If two processes get the same number, the one with the smaller process ID enters first.

So the pair $(number[i], i)$ gives an order for entering the critical section.

Part b — Why deadlock does not happen

Deadlock cannot happen because among all waiting processes, there is always one process with the smallest pair $(number[i], i)$.

That process does not need to wait for any other process with a smaller number, because no such process exists. So it can enter the critical section.

Therefore, the system cannot reach a state where all waiting processes are blocked forever.

Part c — Why mutual exclusion is guaranteed

Suppose two processes are in the critical section at the same time.
Then each one must believe that its pair $(number, id)$ is smaller than the other process's pair.
But this is impossible, because one ordered pair cannot be smaller than the other in both directions at the same time.

Therefore, two processes cannot be in the critical section together.

So Lamport's Bakery Algorithm guarantees mutual exclusion.

5 Final Answers Summary

1. Interleaving problem:

- It is possible to print "x is 10"
- It is possible to print "x is 8"

2. Semaphore solution: Protect the two statements `x = x - 1` and `x = x + 1` with one binary semaphore.

3. Semaphores vs monitors:

- Semaphore: low-level, manual mutual exclusion
- Monitor: high-level, automatic mutual exclusion

4. Message passing: Processes communicate by sending and receiving messages. A single token message can enforce mutual exclusion.

5. Readers/Writers problem: The given solution suffers from writer starvation.

6. Monitor solution: Use a monitor with shared variables and condition variables for readers and writers.

7. Lamport's Bakery Algorithm:

- Works like numbered tickets

- Avoids deadlock
- Guarantees mutual exclusion